Developer Guide

SWAT+ Documentation

docs.swat.tamu.edu

Contents

Pages . . . 2
Generate the build files . . . 2
Pick a compiler . . . 2
Debug vs release . . . 3
Compile and link . . . 3
Install . . . 3
Linux note on libgfortran . . . 3
Required software . . . 4
Starting Visual Studio . . . 4
Cloning the repository . . . 4
Configuring, compiling, running . . . 5
Resources . . . 6
Cost . . . 6
Starting an instance . . . 6
Building . . . 6
Running and debugging . . . 7
Using your own dataset . . . 7
Stopping the instance . . . 8
Deleting the instance . . . 8
How it works . . . 8
Running the tests . . . 9
Adding a new scenario . . . 9
CI . . . 10
How the version is read . . . 10
Listing available tags . . . 10
Building at a specific tag . . . 10
How the version reaches the source . . . 11
Why tag-based versioning . . . 11
Interoperability and portability . . . 12
Readability . . . 12
Robustness . . . 13
Arrays . . . 13
Dynamic memory and pointers . . . 13
Looping . . . 14
Functions and procedures . . . 14
I/O . . . 14
Obsolescent features to avoid . . . 15
Source files . . . 15
References . . . 15

Code Of Conduct **17**
Where work happens . . . 17
Reporting a bug . . . 17
Submitting a change . . . 17
What reviewers look for . . . 17

CI 18
Documentation changes 18

This section is for people working on the SWAT+ Fortran source: building from a clone, running the test suite, and submitting changes back to `swat-model/swatplus`.

If you only need to run a prebuilt SWAT+ executable, see Install instead.

Pages

- Build with CMake. The base out-of-source build flow that every platform uses. Covers configuration, compiler selection (gfortran, ifort, ifx), debug vs release, and installation.
- Visual Studio on Windows. End-to-end workflow for Visual Studio 2022 with the Intel oneAPI Fortran compiler. Cloning, configuring, debugging against an input dataset.
- VS Code Codespaces. Running a SWAT+ build and debug session inside a GitHub Codespaces virtual instance. Useful when you do not want to install a Fortran toolchain locally.
- Testing. The scenario regression test suite that compares model output against golden datasets in `data/`. Driven by `ctest`.
- Tagging and releases. How the build system pulls the version from `git describe` and projects it into `main.f90` and the executable name. Covers the release tag flow.
- Coding conventions. Fortran 90/95 style rules for SWAT+ source contributions: portability, readability, robustness, arrays, I/O, and obsolescent features to avoid.
- Contributing. Fork, branch, pull request, and the CI checks that run on every PR.
- Code of conduct. The Contributor Covenant, applied to the SWAT+ project community.

SWAT+ is built out-of-source. All intermediate and final files go into a separate build directory so the source tree stays clean. The same source tree can host multiple build flavors (different compilers, debug vs release) at the same time.

The build is driven by `CMakeLists.txt` at the repository root. It globs every `.f90` under `src/` and links a single executable.

```
file(GLOB sources src/*.f90)
add_executable(${SWATPLUS_EXE} ${sources})
```

Generate the build files

Make `swatplus` your current working directory.

```
cd <dir>/swatplus
```

Configure the build. This processes `CMakeLists.txt` and writes a build system into `build/`. It does not compile yet.

```
cmake -S . -B build
```

The short form is equivalent.

```
cmake -B build
```

Pick a compiler

Default is `gfortran`. To force it explicitly:

```
cmake -B build -D CMAKE_Fortran_COMPILER=gfortran
```

For Intel Fortran, `ifort` was deprecated in 2024 and removed from the 2025 Intel oneAPI release. Use `ifx` instead.

```
cmake -B build -D CMAKE_Fortran_COMPILER=ifx
```

On Windows, the Intel compilers need a generator toolset:

```
cmake -B build -D CMAKE_GENERATOR_TOOLSET=fortran=ifx
cmake -B build -G "Ninja" -D CMAKE_Fortran_COMPILER=gfortran
```

Note: on Windows, use the default `cmd` shell, not PowerShell, for the command-line build.

Debug vs release

For a debug build with symbols:

```
cmake -B build -D CMAKE_BUILD_TYPE=Debug
```

For a release build:

```
cmake -B build -D CMAKE_BUILD_TYPE=Release
```

Compile and link

From the `swatplus` directory:

```
cmake --build build
```

Parallelize across CPU cores with `-j`. Eight is a reasonable default on most laptops.

```
cmake --build build -j 8
```

The executable lands in `build/` as `swatplus-<version>-<arch>.exe` on Windows and without an extension on Linux and macOS. The version part comes from `git describe --tags`. See Tagging and releases.

To remove all build artifacts and start over:

```
rm -rf build
```

On Windows `cmd`:

```
rd /s /q build
```

To clean just the object files and module files while keeping the build configuration:

```
cmake --build build --target clean
```

Install

Copy the built executable to a directory of your choice with `cmake --install`. The `--prefix` flag sets the base directory.

```
cmake --install build --prefix ~/bin
```

If you omit `--prefix`, CMake installs to the system default binary location. You can also skip this step and copy the executable yourself.

Linux note on libgfortran

If your Linux distribution does not ship the static `libgfortran.a`, remove the `link_libraries("static")` line from `CMakeLists.txt` and build against the dynamic `libgfortran.so`. The resulting binary then needs the dynamic libraries present at runtime. Use `ldd <binary>` to list them.

If you hit a platform-specific compile failure, open an issue on GitHub with your OS, the error output, and any fix you found.

This page covers the full workflow for working on SWAT+ in Visual Studio 2022 on Windows: installing the required tools, cloning the repository, configuring the CMake build, and running a debug session against an input dataset.

Screenshots referenced below live in the SWAT+ source repo's `doc/` folder.

Required software

Install in this order. The Intel compiler requires Visual Studio to be present first.

1. Visual Studio Community 2022. During install, in the **Workloads** panel, check **Desktop development with C++**. That workload pulls in the CMake integration that SWAT+ needs.
2. Intel Fortran Compiler for Windows. Use the standalone installer. The current Intel oneAPI release ships `ifx`. The older `ifort` was deprecated in 2024 and is gone from the 2025 release. SWAT+ produces the same results with either Intel compiler.
3. Git for Windows. Visual Studio bundles its own Git, but SWAT+ uses the standalone `git` during the CMake build to read the current tag for the version string.

Reboot after the three installs finish.

Optional: install Python (from the Microsoft Store) and the Python workload in Visual Studio if you want to run the integrated test suite from inside the IDE.

Starting Visual Studio

The Intel compiler only works with the Visual Studio CMake integration if Visual Studio is launched from the Intel oneAPI command prompt. Follow these steps every time.

1. Open the Windows Start Menu, type `intel`, and select **Intel oneAPI command prompt for Intel 64 for Visual Studio 2022**. Pin it if you use it often.
2. The terminal opens and prints the oneAPI initialization output:

   ```
   :: initializing oneAPI environment...
      Initializing Visual Studio command-line environment...
      Visual Studio version 17.13.0 environment configured.
      "C:\Program Files\Microsoft Visual Studio\2022\Community\"
      Visual Studio command-line environment initialized for: 'x64'
   :  compiler -- latest
   :  debugger -- latest
   :: oneAPI environment initialized ::
   ```

3. At the prompt, run:

   ```
   devenv
   ```

   This launches Visual Studio with the Intel environment variables set.

If you start Visual Studio by clicking the desktop icon directly, the Intel compiler will not be found and the build will fail.

Cloning the repository

Visual Studio can clone the SWAT+ repository and open it as a CMake project in one step.

1. On the Visual Studio start screen, click **Clone a Repository**.
2. Click the GitHub icon (lower left) to browse your repositories. Sign in if prompted. Select your fork of `swatplus` and click **Open**. Adjust the local path if needed.
3. Visual Studio clones the repository and loads it into Solution Explorer as a CMake project. It runs the CMake configure step against whichever Fortran compiler it finds (`ifx`, `ifort`, or `gfortran`). The 2025 Intel compiler only ships `ifx`, so any `ifort` configurations will fail on that version.

Configuring, compiling, running

Configure

SWAT+ is a CMake project. The Visual Studio toolbar exposes the available compiler and build-type combinations through a drop-down. Switching the selection triggers a reconfigure. The Output window prints the CMake settings used, which is the place to verify which compiler picked up.

To force a fresh configure, right-click `CMakeLists.txt` in Solution Explorer and select **Delete Cache and Reconfigure**.

Build

Use **Build > Build All** or **Build > Rebuild All**. The executable name encodes the compiler, architecture, and version. The version is read from the latest Git tag automatically. See Tagging and releases.

The Intel-built Windows executable is dynamically linked. To run it on a machine without Visual Studio, ship the Intel redistributable libraries alongside it.

Run and debug

You need to tell the project which folder contains the input dataset to run against.

1. Right-click `CMakeLists.txt` in Solution Explorer and select **Add Debug Configuration**. If Co-Pilot is hiding Solution Explorer, close that pane first.
2. In the dialog, pick **Default** and click **Select**. Visual Studio creates `launch.vs.json` if it does not exist and opens it for editing.

The default file looks like this:

```
{
  "version": "0.2.1",
  "defaults": {},
  "configurations": [
    {
      "type": "default",
      "project": "CMakeLists.txt",
      "projectTarget": "",
      "name": "CMakeLists.txt"
    }
  ]
}
```

Add a `currentDir` entry pointing at your input dataset. The path can be absolute or relative to the `swatplus` project root. The `name` field is what shows up in the toolbar.

```
{
  "version": "0.2.1",
  "defaults": {},
  "configurations": [
    {
      "type": "default",
      "project": "CMakeLists.txt",
      "projectTarget": "",
      "name": "Ames_testing",
      "currentDir": "C:/work/sp/Ames_testing"
    }
  ]
}
```

Do not change `project` or `type`. You can add more configurations by repeating **Add Debug Configuration** and editing the new block.

Pick your configuration from the toolbar drop-down and run **Debug > Start Debugging** or **Debug > Start Without Debugging**. Breakpoints, conditional watches, and step-through all work.

Resources

- Intel C++ and Fortran Compilers Redistributable Libraries by Version
- Intel Fortran Compiler
- Intel Fortran Compiler for oneAPI release notes
- Visual Studio documentation
- CMake

The SWAT+ repository ships a `.devcontainer/` folder that lets GitHub Codespaces spin up a ready-to-use Visual Studio Code instance in the browser. No local Fortran toolchain needed.

When a Codespaces instance starts, it:

- loads your fork into a remote working folder
- installs the Intel `ifx` and `gfortran` compilers
- configures VS Code to use the CMake build system
- installs the VS Code extensions needed to debug Fortran
- copies the `refdata/Ames_sub1` dataset into `/workdata/Ames_sub1`
- creates an empty `/workdata/my_data` folder for your own inputs

Screenshots referenced below live in the SWAT+ source repo's `doc/` folder.

Cost

GitHub gives every account 120 free CPU hours per month across all Codespaces instances. A 2-CPU instance running for 60 hours uses 120 hours of quota. Beyond that, you pay per active hour. A small per-hour storage fee applies whether the instance is running or stopped.

Instance states:

- **Active.** The instance is in use (interactive or compiling). After 30 minutes of inactivity it goes to **Inactive**. The timeout is configurable.
- **Inactive.** Not running. Storage fee only.
- **Shutdown.** You stopped it manually. Same as Inactive billing-wise.
- **Deleted.** No further charges. A new instance has to be created from scratch to come back.

Starting an instance

1. Sign in to GitHub and go to your `swatplus` fork.
2. Open the hamburger menu (top left) and click **Codespaces**.
3. Click **New codespace** (top right).
4. Pick the repository, the branch, and the CPU count. Two CPUs is enough for SWAT+. Click **Create codespace**.

The instance takes a couple of minutes to come up. You should see an install progress bar in the lower right. When it disappears, a drop-down at the top of the editor asks which Fortran compiler to use. Pick one.

Building

Click the CMake icon in the left sidebar (the triangle-with-wrench) to open the CMake pane.

1. Under **Project Status > Configure**, check the Fortran compiler shown. To change it, hover and click the pencil icon, then pick another.

2. Right-click `CMakeLists.txt` in the CMake pane and select **Clean and Reconfigure**. Do this every time you switch compilers, and any time you add or remove a Fortran source file.
3. Right-click `CMakeLists.txt` again and select **Build All Projects**. The Output panel at the bottom should end with:

   ```
   [build] Build finished with exit code 0
   ```

Running and debugging

Click the debug icon in the left sidebar (the triangle with a bug). Pick `Ames_sub1` from the drop-down and click the green triangle.

After a fresh build, VS Code may ask which executable to run. Any of the debug-build options works. A release build is not debuggable.

Watch model output in the **Terminal** tab at the bottom. Set breakpoints, conditional breakpoints, or just let it run. On a runtime exception, the debugger stops at the failure point and you can inspect variable values.

Using your own dataset

The `/workdata/my_data` folder starts empty. To populate it:

1. In the VS Code Explorer, right-click `/workdata/my_data` and select **Upload**.
2. Allow the cookie prompt if your browser shows one (GitHub needs it to open your local file picker).
3. Select all the files in your input dataset and upload.
4. Switch the debug drop-down to `my_data` and click the green triangle.

To add a third dataset, create a new folder under `/workdata/`, then edit `.vscode/launch.json`. The default file already has blocks for `Ames_sub1` and `my_data`. Copy one block, change `name` and `cwd`, and add a comma between blocks.

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Ames_sub1",
            "type": "cppdbg",
            "request": "launch",
            "program": "${command:cmake.launchTargetPath}",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}/workdata/Ames_sub1",
            "environment": [
                {
                    "name": "PATH",
                    "value": "${env:PATH}:${command:cmake.getLaunchTargetDirectory}"
                }
            ],
            "externalConsole": false,
            "MIMode": "gdb",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        },
```

```
        {
            "name": "walnut_creek",
            "type": "cppdbg",
            "request": "launch",
            "program": "${command:cmake.launchTargetPath}",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}/workdata/walnut_creek",
            "environment": [
                {
                    "name": "PATH",
                    "value": "${env:PATH}:${command:cmake.getLaunchTargetDirectory}"
                }
            ],
            "externalConsole": false,
            "MIMode": "gdb",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        }
    ]
}
```

Stopping the instance

To stop billing for active usage, click **Codespaces** in the bottom-left status bar and select **Stop Current Codespace**. It takes a minute or two.

Deleting the instance

After stopping, go back to the browser tab where you created the instance (or open the Codespaces panel from your GitHub repository). Find the instance at the bottom, click the ellipsis on its row, and select **Delete**. Storage charges stop after deletion. A new instance has to be created from scratch the next time you want to work.

SWAT+ ships a scenario test suite that runs the model against reference input datasets and compares the output files to known-good outputs. The tests catch regressions when source changes shift numeric results outside an accepted tolerance.

Tests are wired into the CMake build and driven by `ctest`.

How it works

The test data lives in `data/`. Each subfolder is a scenario containing input files and the golden output files. The reference output files are committed copies of model output that was reviewed and accepted as correct.

The driver script `test/check.py` does the actual comparison. The wiring sits in `CMakeLists.txt`:

```
find_package(Python REQUIRED)

set(check_py "${PROJECT_SOURCE_DIR}/test/check.py")
set(exe_path "${PROJECT_BINARY_DIR}/${SWATPLUS_EXE}")
set(test_dir "${PROJECT_BINARY_DIR}/data")
set(ref_dir  "${PROJECT_SOURCE_DIR}/data")
```

```
# error tolerances
set(rel_err "0.01")
set(abs_err "1e-8")

add_test(Ithaca_sub6 python3 ${check_py} ${exe_path} ${ref_dir}/Ithaca_sub6 ${test_dir}
 ↪  ${abs_err} ${rel_err})
add_test(Ames_sub1   python3 ${check_py} ${exe_path} ${ref_dir}/Ames_sub1   ${test_dir}
 ↪  ${abs_err} ${rel_err})
```

Two scenarios are registered by default: `Ames_sub1` and `Ithaca_sub6`. Default tolerances are 1% relative error and `1e-8` absolute error. Adding another scenario means adding another `add_test` line that points at a new subfolder in `data/`.

For each scenario, `check.py`:

1. Copies the scenario folder from `data/` into the build directory.
2. Runs the SWAT+ executable inside that copy.
3. Reads `.testfiles.txt` in the scenario, which lists the output file names to check (for example, `wb.txt` and `soc.txt`).
4. For each listed file, compares the new output to the golden copy line by line and column by column. Only floating-point values on matching lines and columns are compared.
5. Two values are considered equal if:

   ```
   abs(v1 - v2) <= abs_err + rel_err * abs(v2)
   ```

6. Prints the count of differing values and the maximum relative and absolute error. The scenario fails if any pair of values does not satisfy the tolerance.

Running the tests

Python 3 has to be on `PATH`. From the `swatplus` directory:

```
cmake -B build
cmake --build build
cd build
ctest
```

`ctest` prints a summary at the end. Detailed per-test logs land in `build/Testing/Temporary/`. The most recent run is captured in `LastTest.log`.

To run a subset:

```
ctest -R Ames_sub1
```

To see test output as it runs:

```
ctest --output-on-failure
```

`make test` from inside `build/` works as an alias.

Adding a new scenario

1. Create a new subfolder under `data/`, for example `data/MyBasin`. Put a full SWAT+ input set in it.
2. Run the model in that folder and review the outputs you want to lock down.
3. Commit those output files alongside the inputs.
4. Add a `.testfiles.txt` listing the output filenames (one per line) that `check.py` should compare.
5. Add an `add_test` line to `CMakeLists.txt`.
6. Reconfigure (`cmake -B build`) and run `ctest`.

CI

The same tests run in GitHub Actions on every push to `main`. See `.github/workflows/test.yml` in the source repository.

A SWAT+ version number is a Git tag on the `main` branch. No source file edits are needed to cut a release. The build system reads the tag and projects it into `src/main.f90` and the executable name.

How the version is read

`CMakeLists.txt` runs `git describe --tags` at configure time. If the current commit is a tagged one, that tag is the version. If not, the output is the most recent reachable tag plus the number of commits since and the abbreviated object name (for example, `61.1-1-g92b4413`).

```
execute_process(
  COMMAND git describe --tags
  WORKING_DIRECTORY ${CMAKE_SOURCE_DIR}
  OUTPUT_VARIABLE TAG
  OUTPUT_STRIP_TRAILING_WHITESPACE
)

set(SWAT_VERSION ${TAG})
set(SWATPLUS_EXE "swatplus-${SWAT_VERSION}.exe")
```

The version becomes part of the executable file name. Building at tag `61.0` produces `swatplus-61.0.exe`. Building one commit past tag `61.1` produces `swatplus-61.1-1-g92b4413.exe`.

Listing available tags

```
git tag
git log
```

Example `git log` output with two tags reachable from `HEAD`:

```
commit 92b44138a3fdb0ada27ed86e2a3a378188f70aef (HEAD -> main, origin/main)
Author: od <odavid@colostate.edu>
Date:   Tue Mar 5 21:43:50 2024 -0700

    fixed 'typo'

commit c5ff4b1083c5b320c72d02ec8b728811bed4441a (tag: 61.1)
...

commit df07e3f572adc21834f05c6711df3566e0231c2f (tag: 61.0)
...
```

Building at a specific tag

Check out the tag, then run the normal CMake build.

```
git checkout 61.0
cmake -B build
cmake --build build
```

The executable will be `swatplus-61.0.exe` (or no extension on Linux and macOS). Switch back to the branch head with `git switch main` and rebuild to get the post-tag version string.

How the version reaches the source

The Fortran source uses a template, `src/main.f90.in`. CMake processes it during configure and writes the final `src/main.f90` with substituted values:

```
prog = " SWAT+ @TODAY@        MODULAR Rev @YEAR@.@SWAT_VERSION@"
    write (*,1000)
    open (9003,file='simulation.out')
    write (9003,1000)
1000 format(1x,"                  SWAT+               ",/,             &
   &          "             Revision @SWAT_VERSION@  ",/,             &
   &          "      Soil & Water Assessment Tool    ",/,             &
   &          "@CMAKE_Fortran_COMPILER_ID@ (@CMAKE_Fortran_COMPILER_VERSION@), @ISO@,
    ↪  @CMAKE_HOST_SYSTEM_NAME@",/,             &
   &          "    Program reading . . . executing",/)
```

The substitution block in `CMakeLists.txt`:

```
if (EXISTS "${PROJECT_SOURCE_DIR}/src/main.f90.in")
    string (TIMESTAMP ISO   "%Y-%m-%d %H:%M:%S")
    string (TIMESTAMP TODAY "%b %d %Y")
    string (TIMESTAMP YEAR  "%Y")

    configure_file(
        "${PROJECT_SOURCE_DIR}/src/main.f90.in"
        "${PROJECT_SOURCE_DIR}/src/main.f90"
    )
endif()
```

After `cmake -B build`, `src/main.f90` contains the resolved values:

```
prog = " SWAT+ Mar 04 2024        MODULAR Rev 2024.61.0.0"

    write (*,1000)
    open (9003,file='simulation.out')
    write (9003,1000)
1000 format(1x,"                  SWAT+               ",/,             &
   &          "             Revision 61.0.0  ",/,             &
   &          "      Soil & Water Assessment Tool    ",/,             &
   &          "GNU (11.4.0), 2024-03-04 14:31:18, Linux",/,             &
   &          "    Program reading . . . executing",/)
```

Do not edit `src/main.f90` directly. It is overwritten every build. Edit `src/main.f90.in` instead.

Why tag-based versioning

- The executable's version always matches the tagged source commit it was built from.
- Version propagation into source and into the binary file name happens automatically.
- GitHub builds source archives and release binaries for each tag through the release workflow in `.github/workflows/build.yml`.

Conventions for writing SWAT+ Fortran source. The goal is portable, readable, and robust code that compiles cleanly on every supported compiler and operating system.

Each item is tagged with one of three levels:

- **Required.** Mandatory. Pull requests that violate these will not be merged.
- **Recommended.** Strongly preferred. Deviations need justification.
- **Encouraged.** Optional. Apply for consistency where reasonable.

The list below is ordered by decreasing impact on code quality. Most subjective style items (indent width, capitalization) sit in the lower tiers.

Interoperability and portability

Required

- Source must conform to the ISO Fortran 95 standard.
- No compiler-specific or platform-specific extensions.
- No use of compiler-dependent error specifier values (`IOSTAT`, `STAT`).
- The source must compile with `gfortran` from GCC, even if the production build uses the Intel compiler.
- `EXIT(N)` is the F90/95 standard exit. Prefer it to `STOP`. `STOP` does not always return an error code. If you need an error code passed back to a script, use `EXIT`, and keep its call sites in one central place.

Recommended

- For floating-point precision and integer size, use the F90/95 `KIND` feature. Do not depend on a vendor-supplied default precision flag.
- Do not use tab characters. Tabs are not part of the Fortran character set and break portability.

Encouraged

- For code that interacts with external frameworks, declare every variable with a `KIND` type to make integration easier.

Readability

Required

- Free-form source.
- Consistent indentation, at least two spaces per level.
- Organize source into modules.
- Do not use Fortran keywords (`DATA`, etc.) as variable names.
- Use meaningful names for variables and parameters. Recognized abbreviations are fine when full names would be unwieldy.
- Every externally-called function or subroutine carries a header describing what it does, its arguments, and the author. Small internal routines can use a few descriptive comments instead.
- No magic numbers. Physical constants (pi, gas constants) go in `PARAMETER` statements, not hard-coded into expressions.
- Do not hard-code numeric literals in argument lists. The compiler's default precision for literals is not guaranteed.
- No `GOTO`. If you must use one, document why.

Recommended

- Name your loops and other constructs (subroutines, functions, modules, interfaces) where the name aids readability, especially in nested loops.
- Comment input, output, and local variables. Grouping comments across similar variables is fine when their names speak for themselves.
- Comment to mark major functional sections.
- Do not use Fortran statement names or intrinsic function names as symbols.
- Prefer named parameters over numeric literals: `REAL, PARAMETER :: PI = 3.14159, ONE = 1.0`.

Encouraged

- In new code, follow your own consistent style. When editing existing code, match the style around you.
- Indent comments the same way as code.

- Group related procedures and data into modules.
- Keep lines to 132 characters or fewer.
- Use `<, >, <=, >=, ==, /=` instead of `.lt., .gt., .le., .ge., .eq., .ne.`.
- Name modules the same as the file that contains them. Avoid putting multiple modules in one file.
- Use blanks around operators, around `=`, and in declarations to separate syntax elements.
- Always use the `::` notation in declarations, even when there are no attributes.
- Line up attributes, variable names, and trailing comments vertically in declarations.
- Remove unused variables.
- Remove debug code once the bug is fixed.

Robustness

Required

- `IMPLICIT NONE` in every program, module, and procedure.
- Use `PRIVATE` in modules by default, then explicitly mark what should be `PUBLIC`. Exception: modules that exist only to expose public data (constants, for example) can stay fully public.
- Initialize every variable. Do not rely on the machine's default.
- Do not initialize a variable of one type with a value of another type.

Recommended

- Do not use `==` or `/=` on floating-point values. Compare against an explicit tolerance instead (an epsilon check).
- In mixed-type expressions and assignments, write the type conversion explicitly. Do not let the compiler do it implicitly.
- No `INCLUDE` files. Use `USE` with modules.
- Derived types belong in their own module, together with the procedures that operate on them.
- Each procedure should do one thing.
- Module-level (global) public variables should hold only static or rarely-changing data.

Encouraged

- Always use parentheses to make expression evaluation order explicit.
- Prefer derived types (`%` field access) over loose collections of arrays.

Arrays

Required

- Subscript expressions must be integer.
- Do not assume any particular array passing mechanism when passing arrays as arguments.

Recommended

- Use array operations and intrinsics where they apply.
- Pass arrays as assumed shape.

Encouraged

- Declare `DIMENSION` on every non-scalar.

Dynamic memory and pointers

Required

- Prefer `ALLOCATABLE` arrays over pointers where possible. Reduces leak and fragmentation risk.

- Pointers are allowed when a subroutine needs to return a declared array to its caller.
- Initialize every pointer to `NULL()` in its declaration: `INTEGER, POINTER :: x => NULL()`.
- Prefer automatic arrays for transient allocations. `ALLOCATABLE` and `POINTER` arrays need explicit `DEALLOCATE`.

Recommended

- Always deallocate. Especially inside subroutines and loops.
- Check the `STAT` argument on `ALLOCATE` and `DEALLOCATE`.
- Do not repeatedly `ALLOCATE`, `DEALLOCATE`, then `ALLOCATE` a larger block. Heap fragmentation follows.

Encouraged

- Dynamic allocation is preferred over fixed-size declarations with worst-case dimensions.
- Use automatic arrays in subroutines for simplicity.

Looping

Required

- No `GOTO` for loop exit or continuation. Use `EXIT` or `CYCLE`.

Recommended

- No numbered `DO` loops (`DO 10 ... 10 CONTINUE`).

Functions and procedures

Required

- Avoid `SAVE`. Hold state in module variables instead.
- No `ENTRY` statements in functions.
- Functions must not return pointers.
- Do not name a user function the same as an intrinsic (`SUM`, etc.).
- A procedure that returns a single value should be a function. The value can be a derived type.
- Communicate through arguments or through the procedure's own module variables.

Recommended

- Every dummy argument that is not a pointer carries an `INTENT`.
- Avoid type-specific intrinsics (`AMAX`, `DMAX`). Use the generic (`MAX`).
- Do not declare static-dimensioned array arguments.
- Validate argument values where it matters.

Encouraged

- On an error, print a message that includes the routine name. You may terminate inside the routine, or pass an error flag back through the argument list.
- Use overloading when the same operation works on multiple types.
- Use modules or `CONTAINS` for explicit interfaces.
- Avoid external routines. They need separate interface blocks that drift out of sync.

I/O

Required

- External-file I/O statements carry `err=`, `end=`, and `iostat=` as appropriate.
- All global variables, if any, are initialized at startup.

Recommended

- Avoid `NAMELIST` I/O.
- Use `WRITE` rather than `PRINT` for non-terminal I/O.
- Use character parameters or in-line format specifiers in `READ` and `WRITE`. Do not use labeled `FORMAT` statements.

Obsolescent features to avoid

Required

- No common blocks. Use modules.
- No assigned or computed `GO TO`. Use `CASE`.
- No arithmetic `IF`. Use a block `IF`.
- Use `REAL`, not `DOUBLE PRECISION`.
- Avoid `DATA`, `ASSIGN`, labeled `DO`, `BACKSPACE`, blank `COMMON`, `BLOCK DATA`.
- Do not branch to an `END IF` from outside its `IF` block.
- No non-integer `DO` loop counters.
- No Hollerith constants.
- No `PAUSE`.
- No multiple `RETURN`, no alternate `RETURN`.

Recommended

- No `EQUIVALENCE`. Use pointers or derived types.

Encouraged

- Do not implicitly reshape arrays when passing them. This was common in F77 and is forbidden by F90 but still possible through external routines without interface blocks.

Source files

Required

- Document every function interface: argument name, type, units, description, constraints, defaults.
- No `INCLUDE`. Use `USE`.
- Keep lines (including comments) under 80 columns where the language allows it.

Recommended

- Keep individual procedures under 300-500 effective lines.
- Separate statement blocks with blank lines or a marker character in column 1.
- Indent consistently.
- Module and subprogram names are lower case. The file is named after the contained module or subprogram, with `.f90`.

Encouraged

- Separate argument declarations from local declarations visually.
- Use descriptive unique names. Keep them under 12-15 characters.
- Indent continuation lines so a multi-line expression lines up readably.
- Comments start with a marker character (`!`). A standalone comment line starts the marker in column 1.

References

- SAS Coding Conventions

- WRF Coding Conventions
- USGS Coding Standards

Code Of Conduct

The SWAT+ project follows the Contributor Covenant, version 2.1.

The full text lives in the source repository as `CODE_OF_CONDUCT.md`. To report a violation, contact the maintainers listed there.

Anyone contributing to SWAT+ (issues, pull requests, code review, mailing-list discussions) is expected to follow it.

Where work happens

- **Code**: https://github.com/swat-model/swatplus
- **Issues**: https://github.com/swat-model/swatplus/issues
- **Pull requests**: https://github.com/swat-model/swatplus/pulls
- **Release notes**: https://github.com/swat-model/swatplus/releases

Reporting a bug

Open an issue with:

- the SWAT+ version (`swatplus -v` or the executable filename)
- the operating system and compiler
- a minimal project that reproduces the problem (or, if possible, a pointer to one of the reference datasets in `refdata/` plus the changes that trigger the bug)
- the contents of `simulation.out`, `diagnostics.out`, and `checker.out` from the failed run.

Bugs that affect water or mass balance, output formats, or input parsing are highest priority.

Submitting a change

1. **Fork** `swat-model/swatplus` on GitHub.
2. **Branch** from `main`. Use a descriptive name (`fix-aquifer-init`, `add-soil-temp-output`).
3. **Build and test locally** before pushing. See Build from source (CMake) and Testing.
4. **Open a pull request** against `swat-model/swatplus:main`. The PR description should state:
   - what the change does
   - why it is needed (link an issue if there is one)
   - which reference datasets you ran and what changed in their outputs
   - any new input or output fields and how existing input files should be updated
5. **Address review comments**. Force-push is acceptable on your branch during review.

What reviewers look for

- Code compiles with both gfortran and ifx (the CI matrix).
- Reference-dataset outputs are unchanged for unrelated runs (no regression).
- New parameters have defaults that preserve existing behaviour when not set.

- Input-file changes are accompanied by SWAT+ Editor changes or a migration note.
- Style follows Coding conventions.

CI

The repository runs GitHub Actions on every pull request. The matrix builds with multiple compilers and runs the scenario tests in `test/`. See `.github/workflows/` for the exact configuration. A red check blocks merging.

Documentation changes

This documentation site lives at https://github.com/swat-model/swatplus-documentation. Documentation pull requests follow the same workflow as code. Every input-reference and output-reference page is verified against the SWAT+ Fortran source; see `SOURCE-OF-TRUTH.md` for the procedure if you are touching one of those pages.